

TECH CHALLENGE — PÓS-GRADUAÇÃO EM ARQUITETURA DE SOFTWARE



Mecânica API

Sistema Integrado de Gestão de Oficinas Mecânicas

Fase 1 + Fase 2 — Entrega Completa

Grupo 14SOAT | Turma SOAT — FIAP

Período: 2025 / 2026

Entrega: Março 2026

Java 21

Spring Boot 3.3

PostgreSQL 16

Arquitetura Hexagonal + DDD

Kubernetes (EKS)

Terraform

GitHub Actions CI/CD

1 052+ Testes

Equipe

Turma 14SOAT — Pós-Graduação em Arquitetura de Software | FIAP 2025/2026



Cleber Fernandes

cleber.fernandes@fiap.com

Software Engineer



Celio Vetrano

celio.vetrano@fiap.com

Software Engineer



Jonnes Nascimento

jones.nascimento@fiap.com

Software Engineer



Thaís Souza

thais.souza@fiap.com

Software Engineer

SOBRE O PROJETO

Este projeto foi desenvolvido como entrega do **Tech Challenge** da Pós-Graduação em Arquitetura de Software da FIAP. O objetivo é demonstrar a aplicação prática de padrões avançados de arquitetura de software — Hexagonal Architecture, Domain-Driven Design, Event-Driven patterns — em um sistema real de gestão de oficinas mecânicas, com infraestrutura cloud-native na AWS.

2

Fases Entregues

4

Membros

31

ADRs Documentados

1052+

Testes Escritos

Visão Geral do Projeto

Problema de negócio, solução técnica e stack adotada

O PROBLEMA

- ▶ Oficinas mecânicas operam com processos manuais e planilhas
- ▶ Ausência de rastreabilidade nas ordens de serviço
- ▶ Aprovação de orçamentos por ligação ou presencialmente
- ▶ Sem controle de estoque integrado ao fluxo de atendimento
- ▶ Dificuldade em gerar relatórios e métricas operacionais
- ▶ Problemas de segurança no acesso às informações de clientes

A SOLUÇÃO

- ▶ Backend completo com API REST documentada via Swagger
- ▶ Máquina de estados para gestão do ciclo de vida das OS
- ▶ Aprovação de orçamentos via link seguro (HMAC-SHA256)
- ▶ Controle de estoque com reserva automática na aprovação
- ▶ Autenticação JWT com controle de acesso por perfil
- ▶ Infraestrutura cloud-native escalável na AWS com EKS

STACK TECNOLÓGICA

Backend

- ▶ Java 21 (LTS) + Records & Sealed Classes
- ▶ Spring Boot 3.3.4 + Spring Security 6
- ▶ Spring Data JPA + Hibernate 6.x
- ▶ JJWT 0.12.5 — tokens JWT seguros
- ▶ Bucket4j — rate limiting
- ▶ Flying Saucer + OpenPDF — geração PDF

Dados & Qualidade

- ▶ PostgreSQL 16 — banco de dados principal
- ▶ Flyway — migrations versionadas (V1-V16)
- ▶ H2 In-Memory — testes unitários
- ▶ TestContainers — testes integrados
- ▶ JaCoCo — cobertura 90% / 80% domínio
- ▶ SonarQube — análise de qualidade

Infra & DevOps

- ▶ Docker multi-stage + distroless
- ▶ Kubernetes (EKS 1.33) + HPA
- ▶ Terraform IaC — VPC, EKS, ECR, ALB
- ▶ GitHub Actions — CI/CD completo
- ▶ Amazon ECR — registry de imagens
- ▶ NGINX Ingress + Cert-Manager

Escopo: API backend RESTful com 13 domínios funcionais, autenticação e autorização completa, domain events, geração de PDF, envio de email, observabilidade (Actuator + métricas) e infraestrutura AWS provisionada por Terraform — tudo com cobertura de testes abrangente.

Core do Domínio

Modelagem DDD, API completa e testes com cobertura >90%

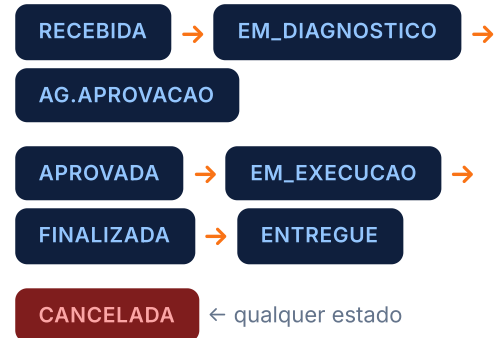
DOMÍNIOS IMPLEMENTADOS

- ▶ **Ordem de Serviço** — agregado raiz com máquina de estados (7 + cancel)
- ▶ **Orçamento** — ciclo gerado → aprovado/rejeitado + link seguro
- ▶ **Estoque** — controle de peças e insumos com reservas
- ▶ **Clientes & Veículos** — cadastro com CPF, CNPJ, telefone
- ▶ **Mecânicos & Atendentes** — gestão de usuários por perfil
- ▶ **Catálogo** — Peças, Insumos e Serviços
- ▶ **Priorização de OS** — fila operacional ordenada
- ▶ **Relatórios** — tempo médio, PDF de OS finalizada

SEGURANÇA & AUTORIZAÇÃO

- ▶ JWT com JJWT 0.12.5 — autenticação stateless
- ▶ Roles: Admin, Mecânico, Atendente, Cliente
- ▶ `@PreAuthorize` + `OsSecurity` — autorização híbrida (ADR-010)
- ▶ Rate limiting via Bucket4j (10 req/min)
- ▶ Secrets gerenciados via K8s Secrets (produção)

MÁQUINA DE ESTADOS — ORDEM DE SERVIÇO



QUALIDADE DE CÓDIGO

90%

Cobertura Global

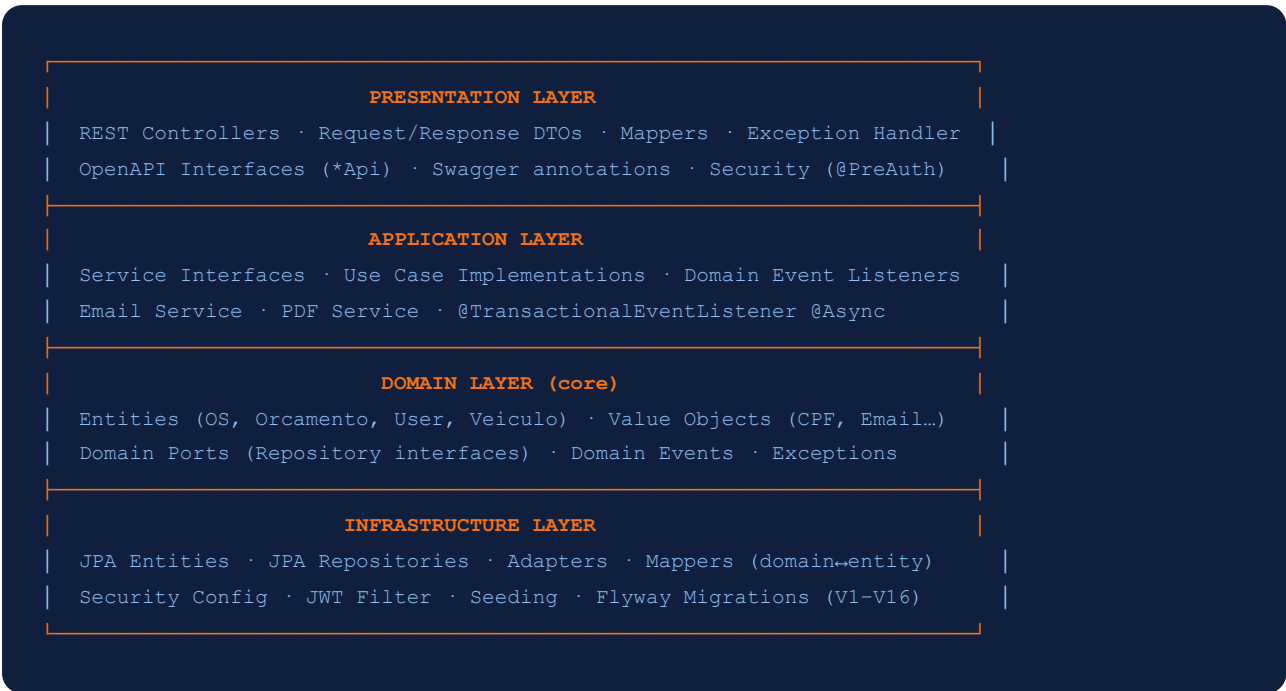
80%

Cobertura Domínio

- ▶ Checkstyle — Google Java Style Guide
- ▶ PMD — complexidade ciclomática e código morto
- ▶ SpotBugs — null dereference e resource leaks
- ▶ SonarQube — análise contínua de qualidade
- ▶ Spotless — auto-formatação (pre-commit hook)
- ▶ Allure Reports — relatórios de teste enriquecidos

Arquitetura Hexagonal + DDD

Ports & Adapters com Domain-Driven Design — 282 classes Java



REGRA DE DEPENDÊNCIA

- ▶ Setas sempre apontam **domínio** (nunca o contrário)
- ▶ Domain layer: zero anotações Spring, zero JPA — POJO puro
- ▶ Ports: interfaces Java no domínio, implementações na infra
- ▶ Adapters JPA implementam as interfaces de repositório do domínio
- ▶ Dois `infra/mapper/` e `presentation/mapper/` mappers:

BOUNDED CONTEXTS

Contexto	Responsabilidade
Atendimento	OS, Clientes, Veículos
Diagnóstico	Orçamentos, Mecânicos
Estoque	Peças, Insumos, Serviços
Integração	API M2M, Email, PDF

DOMAIN EVENTS (SPRING APPLICATIONEVENT)

- ▶ `OsCriadaEvent` — notifica criação
- ▶ `OrcamentoGeradoEvent` — envia email ao cliente
- ▶ `OrcamentoAprovadoEvent` — reserva estoque
- ▶ `OrdemServicoAguardandoAprovacaoEvent`
- ▶ `OsFinalizadaEvent` — gera PDF de entrega
- ▶ `OrdemServicoCanceladaEvent` — estorna estoque
- ▶ Listeners `@Async` + `@TransactionalEventListener` com

► OrcamentoReprovadoEvent —
cancela
reservas

Novos Endpoints & Containerização

APIs de integração, segurança aprimorada e Docker multi-stage

NOVOS ENDPOINTS — FASE 2

Método	Endpoint	Auth	Descrição
POST	/api/ordens-servico/abertura-completa	JWT — Atendente	Cria OS com veículo, serviços e mecânico em um único request
GET	/api/ordens-servico/fila-operacional	JWT — Mecânico	Fila priorizada de OS em execução por mecânico autenticado
GET	/api/ordens-servico/{id}/status	JWT — Público role	Consulta status resumido da OS (para clientes)
POST	/api/integracoes/orcamentos/aprovacao	API Key M2M	Aprovação de orçamento por integração máquina-a-máquina

EMAIL COM LINK SEGURO (HMAC-SHA256)

- ▶ Ao gerar orçamento, email automático é enviado ao cliente
- ▶ Link único com token HMAC-SHA256 assinado com segredo
- ▶ Token contém ID do orçamento + timestamp de expiração
- ▶ Validade configurável (padrão: 48h)
- ▶ Endpoint público para aprovação: nenhuma senha necessária
- ▶ Implementado `HmacUtils` + Thymeleaf via

DOCKER MULTI-STAGE BUILD

- ▶ **Dockerfile** — multi-stage com JRE Alpine (imagem mínima ~120MB)
- ▶ **Dockerfile.deploy** — distroless para produção (zero shell)
- ▶ Stage 1: Maven build com cache de dependências
- ▶ Stage 2: apenas JRE + artefato compilado
- ▶ Usuário não-root em todas as imagens (segurança)
- ▶ Trivy scan integrado no pipeline de build

DOCKER COMPOSE — 5 SERVIÇOS

mecanica-app :8080
API principal + Swagger UI

mecanica-postgres :5433
PostgreSQL 16

mecanica-adminer :8081
Browser de banco de dados

Infraestrutura AWS

Kubernetes (EKS 1.33) + Terraform IaC + HPA para escalabilidade automática



HPA — AUTO SCALING

- ▶ CPU: **70%** threshold
- ▶ Memória: **80%** threshold
- ▶ Min replicas: **1**
- ▶ Max replicas: **10**
- ▶ Metrics Server instalado

TERRAFORM (IAC)

- ▶ VPC + subnets + NAT Gateway
- ▶ EKS 1.33 + node group t3.large
- ▶ Amazon ECR (repositório privado)
- ▶ ALB Controller via Helm
- ▶ NGINX Ingress + Cert-Manager

KUBERNETES MANIFESTS

- ▶ Namespace + ConfigMap
- ▶ Secrets (JWT, DB, API Key)
- ▶ Deployment + Service
- ▶ HPA + PVC (EBS 10Gi)
- ▶ Overlays: lab / prod

Pipeline CI/CD — GitHub Actions

5 workflows orquestrados — do push ao deploy em EKS



WORKFLOWS

Workflow	Trigger	Arquivo	Responsabilidade
CI	Push / PR → develop	ci.yml	Compilação Maven, cache M2, lint (Checkstyle/PMD/SpotBugs)
Test	Push / PR → develop	test.yml	Testes unitários e integrados (TestContainers), cobertura JaCoCo
Build	Merge → main	build.yml	Docker multi-stage + Trivy scan + push para ECR
CD	Após build	cd.yml	Deploy em EKS via kubectl apply + k8s-deploy.sh (envsubst)
Infra	Manual dispatch	infra.yml	Terraform plan/apply — provisionamento da infraestrutura AWS

SEGURANÇA NO PIPELINE

- ▶ Secrets via GitHub Actions Secrets (nunca expostos em logs)
- ▶ OIDC para autenticação AWS — sem chaves de longa duração
- ▶ Trivy: scan de vulnerabilidades na imagem Docker
- ▶ EBS CSI Driver com OIDC para volumes persistentes
- ▶ Token Sonar removido do código (ADR-SEC-001)

QUALIDADE NO PIPELINE

- ▶ Cache Maven: build ~40% mais rápido
- ▶ JaCoCo: falha se cobertura < 90% global
- ▶ SonarQube analysis em cada merge
- ▶ Pre-commit hook: Spotless auto-format
- ▶ 3 índices PostgreSQL em V16 (status, data)

Requisitos vs Entregáveis

Mapeamento completo dos 22 requisitos das Fases 1 e 2

ID	Requisito	Status	Evidência / Observação
FASE 1 — Core do Domínio			
F1-01	API REST para gerenciamento de OS com DDD	✓ OK	13 domínios, agregados, VOs, ports & adapters
F1-02	Máquina de estados para OS	✓ OK	7 estados + CANCELADA, transições validadas
F1-03	Gestão de orçamentos com aprovação do cliente	✓ OK	Link HMAC-SHA256, email Thymeleaf, fluxo invertido (ADR-022)
F1-04	Controle de estoque (peças e insumos)	✓ OK	Reserva na aprovação de orçamento (ADR-019)
F1-05	Autenticação e autorização por perfil	✓ OK	JWT + roles + @PreAuthorize + OsSecurity (ADR-010)
F1-06	Cadastro de clientes, veículos e usuários	✓ OK	CPF/CNPJ validados, placa veículo, endereço, telefone
F1-07	Catálogo de serviços, peças e insumos	✓ OK	CRUD completo com paginação (Pageable, max 100)
F1-08	Relatórios operacionais	✓ OK	Tempo médio de serviços, PDF de OS finalizada
F1-09	Cobertura de testes $\geq 90\%$ (global) / 80% (domínio)	✓ OK	1052+ testes, JaCoCo gate ativo no CI
F1-10	Documentação Swagger / OpenAPI	✓ OK	15 interfaces *Api, Swagger em /swagger-ui.html
F1-11	Collection Insomnia exportada	✓ OK	docs/api/Insomnia_export.yaml com fluxo E2E
FASE 2 — Infraestrutura e Escalabilidade			
F2-01	Novos endpoints (abertura completa, fila, status, integração)	✓ OK	4 endpoints implementados e documentados
F2-02	Docker — containerização da aplicação	✓ OK	Dockerfile multi-stage Alpine + distroless deploy
F2-03	Docker Compose com todos os serviços	✓ OK	app + postgres + adminer + mailhog + sonarqube

F2-04	Kubernetes — manifests e deploy	✓ OK	k8s/base/ + k8s/overlays/ (lab, prod)
F2-05	HPA — escalabilidade automática	✓ OK	CPU 70% / Mem 80%, min 1, max 10 réplicas
F2-06	Banco de dados em K8s com persistência	⚠ PVC	PostgreSQL StatefulSet + PVC 10Gi (EBS) — não RDS (custo)
F2-07	Infraestrutura como Código (Terraform)	✓ OK	VPC, EKS 1.33, ECR, ALB, NGINX, Cert-Manager via Terraform
F2-08	Pipeline CI/CD automatizado	✓ OK	5 GitHub Actions workflows: ci, test, build, cd, infra
F2-09	README completo com arquitetura e instruções	✓ OK	README.md com diagrama ASCII AWS, make commands, guias
F2-10	PDF de entrega com todos os entregáveis	✓ OK	Este documento — docs/entrega/entrega-grupo14soat.pdf
F2-11	Vídeo demonstrativo (≤ 15 min)	✗ Pendente	A ser gravado pela equipe

Nota F2-06: Optamos por PostgreSQL no Kubernetes (StatefulSet + PVC) em vez de RDS por custo de laboratório. A arquitetura está preparada para migração a RDS alterando apenas a string de conexão e secrets.

Decisões Arquiteturais (ADRs)

31 Architecture Decision Records documentados — Fase 1: 28 | Fase 2: 3

FASE 1 — ADRS 001-028

ADR	Decisão
ADR-001	PostgreSQL como banco principal
ADR-002	Validação na camada de aplicação
ADR-003	API baseada em tarefas para status de OS
ADR-004	Reorganização de controllers de OS
ADR-005	Serviços adicionais em aprovação
ADR-006	Unificação de validação de regras
ADR-007	Reforço de validação de domínio
ADR-008	Padronização de exceções (código de erro)
ADR-009	Implementação do SonarQube
ADR-010	Estratégia de autorização híbrida ⭐
ADR-011	Gestão de estoque de peças
ADR-012	Sistema de priorização de OS
ADR-013	Automação de correção de warnings
ADR-014	Gestão automática de orçamentos
ADR-015	Geração e armazenamento de PDFs
ADR-016	Sistema de envio de emails
ADR-017	Fluxo de uso com Insomnia
ADR-018	Estratégia de testes E2E

FASE 2 — ADRS 029-031

ADR	Decisão
ADR-029	Estratégia de testes de carga (k6)
ADR-030	Separação do pipeline CI/CD
ADR-031	Decisões consolidadas Fase 2 ⭐

ADRS DE MAIOR IMPACTO ⭐

ADR-010 — Autorização Híbrida

JWT roles + segurança a nível de objeto (OsSecurity) — clientes só acessam suas próprias OS

ADR-019 — Estoque Desacoplado

Estoque deduzido na aprovação do orçamento, não do budget — evita inconsistências

ADR-022 — Fluxo Invertido OS/Orçamento

Cliente aprova orçamento → OS avança para APROVADA (não o contrário)

ADR-031 — Decisões Fase 2

PostgreSQL em PVC vs RDS, OIDC para EBS, pipeline separado CI vs CD

ADR-019	Desacoplamento Orçamento/Estoque 🌟
ADR-020	Sub-domínio de prioridade de OS
ADR-021	Flexibilização de atribuição de mecânico
ADR-022	Inversão do fluxo OS/Orçamento 🌟
ADR-023	Monitoramento de tempo médio de serviços
ADR-024	Endpoint de tempo médio de OS
ADR-025	Arquitetura de logging
ADR-026	Reprovação de orçamento e impacto na OS
ADR-027	Estratégia de testes integrados
ADR-028	Reestruturação da documentação

Links de Entrega

Todos os artefatos e recursos do projeto

REPOSITÓRIO GITHUB

<https://github.com/clefern/fiap-tc-mecanica-java>

Código-fonte completo, histórico de commits, PRs, pipelines CI/CD e documentação

SWAGGER UI — DOCUMENTAÇÃO DA API

<http://localhost:8080/swagger-ui.html>

OpenAPI 3.0 — todos os 50+ endpoints documentados com exemplos de request/response

INSOMNIA COLLECTION

[docs/api/Insomnia_export.yaml](#)

Collection exportada com fluxo E2E completo e endpoints por categoria — importar no Insomnia

DOCUMENTAÇÃO COMPLETA

README.md + docs/ (31 ADRs + arquitetura + infraestrutura + qualidade)

Toda a documentação arquitetural, instruções de execução e decisões técnicas



Vídeo Demonstrativo — PLACEHOLDER

O link do vídeo será adicionado aqui após a gravação.

Para adicionar: editar `presentation.html`, localizar `<!-- VIDEO_LINK_PLACEHOLDER -->` e substituir este bloco pelo link real. Executar `gerar-pdf.sh` para regenerar.

Conteúdo esperado: deploy na AWS, demo das APIs (Swagger), escalabilidade via HPA, pipeline CI/CD — duração máxima 15 minutos.

COMO EXECUTAR LOCALMENTE

- ▶ `make dev` — inicia todos os serviços Docker
- ▶ `make test` — roda testes unitários
- ▶ `make integration-test` — testes com DB real
- ▶ `make coverage` — relatório JaCoCo
- ▶ Swagger disponível em: `localhost:8080/swagger-ui.html`

USUÁRIOS DE TESTE (SEED)

Perfil	Email	Senha
Admin	admin@mecanica.com	123456
Mecânico	mecanico@mecanica.com	123456
Atendente	atendente@mecanica.com	123456
Cliente	cliente@mecanica.com	123456